



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Recorridos Inorden, Preorden, Postorden y BFS
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero - B
Alumno:	Ortiz Cunalata Alina Elizabeth
Asignatura:	Estructura de datos
Docente:	Ing. José Caiza, Mg.

I. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

Objetivo General:

Implementar y analizar los principales recorridos de árboles binarios utilizando C++ y Java, aplicando estructuras de datos dinámicas, recursividad y colas.

Objetivos Específicos:

- Analizar el funcionamiento de los árboles binarios y sus distintos tipos de recorrido.
- Desarrollar recorridos recursivos en profundidad: Preorden, Inorden y Postorden.
- Utilizar una cola para implementar el recorrido por amplitud (BFS).
- Identificar las diferencias entre las implementaciones realizadas en C++ y Java.
- Relacionar los recorridos de árboles binarios con una aplicación práctica en un sistema web.

2.2 Desarrollo Teórico

Introducción

Un recorrido de árbol es un proceso sistemático que permite visitar cada uno de los nodos que componen una estructura jerárquica exactamente una vez. Los árboles binarios son estructuras de datos no lineales ampliamente utilizadas en informática para representar relaciones jerárquicas como sistemas de archivos, expresiones matemáticas, estructuras organizacionales y módulos de software.

La importancia de los recorridos radica en que permiten procesar la información almacenada en el árbol de diferentes maneras, dependiendo del orden en que se visitan los nodos. En esta práctica se estudian cuatro tipos fundamentales de recorridos: Inorden, Preorden, Postorden (técnicas de búsqueda en profundidad o DFS) y BFS (búsqueda en amplitud o nivel por nivel).

Marco Teórico

Un árbol binario es una estructura de datos jerárquica en la cual cada nodo puede tener como máximo dos hijos: uno izquierdo y uno derecho. Se utiliza ampliamente en informática para representar relaciones jerárquicas, búsquedas eficientes y organización de información.

Recorridos de Árboles Binarios

Los recorridos son métodos sistemáticos para visitar todos los nodos de un árbol.

1. Recorrido Inorden (Izquierda → Raíz → Derecha)



Se aplica en Árboles Binario de Búsqueda (BST), produce los elementos en orden ascendente. Es útil para ordenar datos y para obtener listados secuenciales.

2. Recorrido Preorden (Raíz → Izquierda → Derecha)

Es útil para mostrar la estructura jerárquica de un sistema, como menús o módulos, ya que primero se visita el nodo principal y luego sus submódulos.

3. Recorrido Postorden (izquierda → Derecha → Raíz)

Se aplica cuando es necesario procesar primero los nodos hijos antes que el padre, por ejemplo, al eliminar un árbol o calcular dependencias.

4. Recorrido BFS (Breadth-First Search):

Se visitan todos los nodos nivel por nivel, de izquierda a derecha. Utiliza una cola (estructura FIFO) para almacenar los nodos pendientes de procesar. Es ideal para mostrar información en capas o niveles jerárquicos.

Comparación DFS vs BFS [3] [4]:

Característica	DFS (Profundidad)	BFS (Amplitud)
Estructura auxiliar	Pila (implícita por recursividad)	Cola (explícita)
Orden de visita	Va hasta el fondo antes de retroceder	Va nivel por nivel
Uso de memoria	O(altura del árbol)	O(ancho del árbol)

2.3 Desarrollo y Ejercicios Resueltos

Ejercicio 1: Recorridos del árbol inicial.

```
      10
     /  \
    5    15
   / \   / \
  2  7 12 20
```

Preorden : 10, 5, 2, 7, 15, 12, 20

Inorden : 2, 5, 7, 10, 12, 15, 20

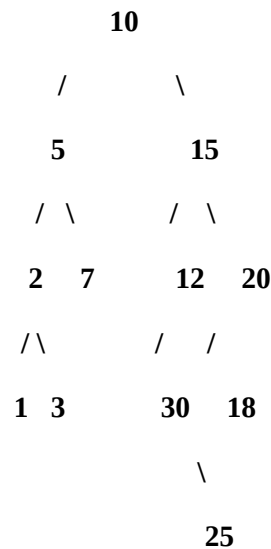
Postorden: 2, 7, 5, 12, 20, 15, 10

BFS (Nivel por nivel): 10, 5, 15, 2, 7, 12, 20

Ejercicio 2: Árbol modificado con nodos 1, 3, 18 y 25, 30



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: MARZO – JULIO 2025



Preorden: 10, 5, 2, 1, 3, 7, 15, 12, 30, 20, 18, 25

Inorden: 1, 2, 3, 5, 7, 10, 12, 15, 18, 20, 25, 30

Postorden: 1, 3, 2, 7, 5, 30, 12, 25, 18, 20, 15, 10

BFS: 10, 5, 15, 2, 7, 12, 20, 1, 3, 30, 18, 25

Ejercicio 3: Función para contar nodos.

Explicación del algoritmo: La función utiliza recursividad. Por cada nodo, suma 1 (el nodo actual) más el resultado de contar los nodos del subárbol izquierdo y del subárbol derecho. Cuando llega a un nodo nulo, retorna 0.

Código en c++:

```
int contarNodos(Nodo* raiz) {    if (raiz == nullptr) return 0;    return  
1 + contarNodos(raiz->izquierda) + contarNodos(raiz->derecha); }  
  
// Ejemplo de uso cout << "Total de nodos: " <<  
contarNodos(raiz) << endl;
```

Código en java:

```
public static int contarNodos(Nodo raiz) {    if (raiz == null) return  
0;    return 1 + contarNodos(raiz.izquierda) +  
contarNodos(raiz.derecha); }  
  
// Ejemplo de uso  
System.out.println("Total de nodos: " + contarNodos(raiz));
```

Ejercicio 4: Función para contar hojas.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: MARZO – JULIO 2025



Explicación del algoritmo: La función recorre el árbol recursivamente. Si un nodo es nulo, retorna 0. Si un nodo no tiene hijo NI derecho, es una hoja y retorna 1. En caso contrario, suma el resultado de contar hojas en ambos subárboles.

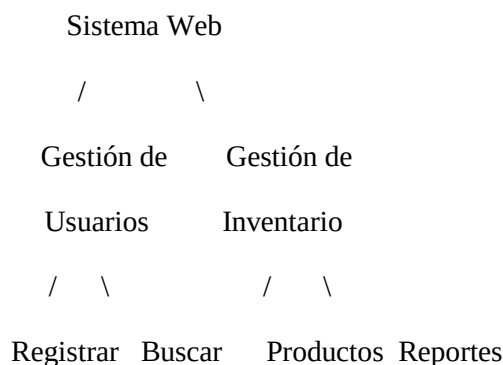
Código en c++:

```
int contarHojas(Nodo* raiz) {    if (raiz == nullptr) return 0;    if  
(raiz->izquierda == nullptr && raiz->derecha == nullptr) return 1;  
return contarHojas(raiz->izquierda) + contarHojas(raiz->derecha);  
}  
  
// Ejemplo de uso cout << "Total de hojas: " <<  
contarHojas(raiz) << endl;
```

Código en java:

```
public static int contarHojas(Nodo raiz) {    if (raiz == null)  
return 0;    if (raiz.izquierda == null && raiz.derecha == null)  
return 1;        return  contarHojas(raiz.izquierda)  +  
contarHojas(raiz.derecha);  
}  
  
// Ejemplo de uso  
System.out.println("Total de hojas: " + contarHojas(raiz));
```

Ejercicio 5: Representación del sistema web como árbol binario.



- **Menú principal:** Se usa Preorden, ya que primero se muestra el nodo raíz (Sistema Web) y luego sus submódulos.
- **Procesar módulos internos:** Se usa Postorden, porque primero se procesan los hijos antes que el padre.
- **Mostrar nivel por nivel:** Se usa BFS, ya que permite recorrer los módulos por capas jerárquicas.



2.4 Resultados y Capturas

Ejercicio 1: Recorridos manuales del árbol base

Ejecución:

Java

```
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_
clipse Adoptium\jdk-21.0.11-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages' -cp 'C:\Users\Usa
e\ccdc4c7e3c835599ca497e100971757c0\rsdhat.java\jdk_ws\tarea2.2 repo_recorridos_arboles_uta_89e9c8b1\bin' 'Main'

=====
EJERCICIO 1 - ÁRBOL INICIAL
=====
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
```

C++

```
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta> cd src\cpp
PS C:\Users\usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta\src\cpp> g++ main.cpp
o main.exe
PS C:\Users\Usuario\OneDrive\Documentos\Universidad\TERCER SEMESTRE\Estructura de Datos\PARCIAL - 2\tarea2.2 repo_recorridos_arboles_uta\src\cpp> .\main.exe

=====
EJERCICIO 1 - ÁRBOL INICIAL
=====
Preorden: 10 5 2 7 15 12 20
Inorden: 2 5 7 10 12 15 20
Postorden: 2 7 5 12 20 15 10
BFS: 10 5 15 2 7 12 20
```

Ejercicio 2: Árbol ampliado (nodos 1, 3, 18, 25, 30)

Ejecución:

Java

```
=====
EJERCICIO 2 - ÁRBOL CON NODOS AGREGADOS
Nodos agregados: 1, 3, 18, 25 + nodo extra 30
=====
Preorden: 10 5 2 1 3 7 15 12 30 20 18 25
Inorden: 1 2 3 5 7 10 12 30 15 18 25 20
Postorden: 1 3 2 7 5 30 12 25 18 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 30 18 25
```

C++

```
=====
EJERCICIO 2 - ÁRBOL CON NODOS AGREGADOS
Nodos agregados: 1, 3, 18, 25, 30
=====
Preorden: 10 5 2 1 3 7 15 12 30 20 18 25
Inorden: 1 2 3 5 7 10 12 30 15 18 25 20
Postorden: 1 3 2 7 5 30 12 25 18 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 30 18 25
```



Ejercicio 3: Función para contar nodos.

Ejecución:

Java

```
=====
EJERCICIO 3 - CONTAR NODOS
=====
Total de nodos en el árbol: 12
```

C++

```
=====
EJERCICIO 3 - CONTAR NODOS
=====
Total de nodos en el arbol: 12
```

Ejercicio 4: Función para contar hojas.

Ejecución:

Java

```
=====
EJERCICIO 4 - CONTAR HOJAS
=====
Total de hojas en el arbol: 5
(Hojas: nodos sin hijos: 1, 3, 7, 25, 30)
```

C++

```
=====
EJERCICIO 4 - CONTAR HOJAS
=====
Total de hojas en el arbol: 5
(Hojas: nodos sin hijos: 1, 3, 7, 25, 30)
```

Ejercicio 5: Representación del sistema web como árbol binario.

Ejecución:

Java



```
jdk-21.0.11.10-hotspot\bin\java.exe -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\Usuario\AppData\Local\Temp\code\scad97e100971757c6\redhat.java\jdk_ues\tarea2.2_repo_recorridos_arboles_uta_09e9c8b1\bin Main

EJERCICIO 5 - SISTEMA WEB
Aplicación al proyecto final

Estructura del Sistema Web:
      Sistema Web
     /      \
Gestión de  Gestión de
Usuarios    Inventario
 /  \      /  \
Registrar Buscar Productos Reportes

--- RESULTADOS DE RECORRIDOS EN EL SISTEMA WEB ---
NOTA: Los resultados mostrados corresponden a la estructura numerica
aplicando la misma logica a los modulos del sistema web:
- Preorden: Sistema Web -> Gestion Usuarios -> Registrar -> Buscar -> Gestion Inventario -> Productos -> Reportes
- Postorden: Registrar -> Buscar -> Gestion Usuarios -> Productos -> Reportes -> Gestion Inventario -> Sistema Web
- BFS: Sistema Web -> Gestion Usuarios -> Gestion Inventario -> Registrar -> Buscar -> Productos -> Reportes
```

C++

```
EJERCICIO 5 - SISTEMA WEB
Aplicación al proyecto final

Estructura del Sistema Web:
      Sistema Web
     /      \
Gestion de  Gestion de
Usuarios    Inventario
 /  \      /  \
Registrar Buscar Productos Reportes

--- RESULTADOS DE RECORRIDOS EN EL SISTEMA WEB ---
NOTA: Los resultados mostrados corresponden a la estructura numerica
aplicando la misma logica a los modulos del sistema web:
- Preorden: Sistema Web -> Gestion Usuarios -> Registrar -> Buscar -> Gestion Inventario -> Productos -> Reportes
- Postorden: Registrar -> Buscar -> Gestion Usuarios -> Productos -> Reportes -> Gestion Inventario -> Sistema Web
- BFS: Sistema Web -> Gestion Usuarios -> Gestion Inventario -> Registrar -> Buscar -> Productos -> Reportes
```

2.5 Preguntas de Reflexión

1. ¿Qué recorrido sirve para ordenar valores en un BST?

El recorrido Inorden es el que sirve para ordenar valores en un Árbol Binario de Búsqueda (BST), ya que visita primero el subárbol izquierdo (valores menores), luego la raíz y finalmente el subárbol derecho (valores mayores), produciendo una secuencia ascendente de todos los elementos almacenados en el árbol.

2. ¿Qué diferencia existe entre DFS y BFS?

DFS (Preorden, Inorden, Postorden) explora en profundidad usando recursividad, mientras que BFS explora nivel por nivel usando una cola. En el sistema web, DFS iría hasta "Registrar" antes de ver "Inventario", mientras que BFS mostraría primero "Sistema Web", luego "Usuarios" e "Inventario", y después "Registrar", "Buscar", "Productos" y "Reportes".

3. ¿Por qué BFS requiere una cola?

BFS requiere una cola porque necesita procesar los nodos en orden FIFO, asegurando que los nodos del nivel actual se recorran completamente antes de pasar al siguiente nivel, lo que permite mostrar el sistema web nivel por nivel: Nivel 0 → Sistema Web, Nivel 1 → Usuarios e Inventario, Nivel 2 → Registrar, Buscar, Productos y Reportes.



4. ¿En qué caso real se puede usar Preorden?

Preorden es ideal para mostrar un menú principal con estructura jerárquica completa, como el menú de navegación del sistema web: Sistema Web → Usuarios → Registrar → Buscar → Inventario → Productos → Reportes. Esto permite al usuario ver primero el módulo principal y luego los submódulos en orden de aparición.

5. ¿En qué caso real se puede usar Postorden?

Postorden es útil para procesar módulos internos antes que el módulo principal, como al cerrar sesión: primero cerrar submódulos (Registrar, Buscar, Productos, Reportes), luego módulos principales (Usuarios, Inventario) y finalmente el Sistema Web. Esto garantiza que todos los procesos internos finalicen correctamente antes de salir del sistema principal.

2.6 Preguntas tipo Moodle

1. ¿Cuál es el orden del recorrido Inorden?

- A. Raíz, izquierda, derecha
- B. Izquierda, raíz, derecha
- C. Izquierda, derecha, raíz
- D. Nivel por nivel **Respuesta: B**

2. ¿Qué estructura utiliza BFS?

- A. Pila
- B. Lista circular
- C. Cola
- D. Árbol AVL **Respuesta: C**

3. ¿Cuál recorrido visita primero la raíz?

- A. Inorden
- B. Preorden
- C. Postorden
- D. BFS únicamente **Respuesta: B**

4. ¿Cuál recorrido procesa la raíz al final?

- A. Preorden
- B. Inorden
- C. Postorden
- D. Nivel por nivel **Respuesta: C**

5. En un BST, el recorrido Inorden permite obtener:

- A. Elementos desordenados
- B. Elementos por niveles
- C. Elementos en orden ascendente
- D. Solo hojas **Respuesta: C**

Pregunta práctica:



Complete el código C++ del recorrido inorden:

```
void inorden(Nodo* raiz) {    if (raiz == nullptr)

return;    inorden(raiz->izquierda); // Subárbol

izquierdo        cout << raiz->dato << " ";

inorden(raiz->derecha);    // Subárbol derecho

}
```

Respuesta: inorden

2.7 Comparación C++ vs Java

En esta sección se analizan las principales diferencias entre las implementaciones realizadas en C++ y Java para los recorridos de árboles binarios, considerando aspectos como sintaxis, manejo de memoria, estructuras de datos y rendimiento.

Aspecto	C++	Java
Declaración de nodos	Uso de struct o class con punteros (*)	Uso de <code>class</code> con referencias implícitas
Punteros	Explícitos (*, ->, &, nullptr)	Inexistentes (se usan referencias automáticas)
Manejo de memoria	Manual (new / delete)	Automático (Garbage Collector)
Recursividad	Soporte nativo, riesgo de desbordamiento de pila	Soporte nativo, optimizado por JVM
Cola para BFS	<code>#include <queue></code> usando <code>queue<Nodo*></code>	<code>import java.util.Queue</code> usando <code>LinkedList<>()</code>
Sintaxis de impresión	<code>cout << valor << " ";</code>	<code>System.out.print(valor + " ");</code>
Compilación	<code>g++ main.cpp -o programa</code>	<code>javac Main.java</code>
Ejecución	<code>./programa</code> (Linux) o <code>programa.exe</code> (Windows)	<code>java Main</code>
Velocidad de ejecución	Más rápido (código nativo)	Más lento (bytecode + JIT)
Portabilidad	Requiere recompilación por plataforma	Bytecode multiplataforma (JVM)
Depuración	Herramientas como GDB	Depurador integrado en IDE (Eclipse, VS Code, IntelliJ)

Comparación de código: Creación de un nodo

Lenguaje	Ventajas	Desventajas
----------	----------	-------------



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: MARZO – JULIO 2025



C++	• Mayor velocidad de ejecución • Control total sobre la memoria • Ideal para sistemas embebidos o de alto rendimiento • Menor consumo de recursos	• Gestión manual de memoria (propensa a errores) • Sintaxis más compleja (punteros) • Menos portabilidad • Mayor riesgo de memory leaks
Java	• Gestión automática de memoria (GC) • Sintaxis más limpia y simple • Alta portabilidad (JVM) • Menor riesgo de errores de memoria • Gran cantidad de librerías estándar	• Menor rendimiento que C++ • Mayor consumo de RAM • Mayor tiempo de inicio (JVM) • Latencia por pausas del GC

2.8 Conclusiones

- Los recorridos de árboles binarios permiten organizar y acceder a la información de diferentes maneras según la necesidad del sistema.
- Los recorridos DFS (Preorden, Inorden y Postorden) utilizan recursividad para recorrer los nodos en profundidad, mientras que BFS recorre el árbol por niveles usando una cola.
- La implementación en C++ y Java permitió comprender el manejo de estructuras dinámicas, punteros, objetos y recursividad en ambos lenguajes.
- Las funciones adicionales como contar nodos, contar hojas y calcular altura ayudan a analizar la estructura y eficiencia del árbol binario.

2.9 Recomendaciones

- Practicar distintos tipos de recorridos para comprender mejor el comportamiento de los árboles binarios.
- Implementar árboles binarios de búsqueda para mejorar la organización y búsqueda de datos.
- Realizar pruebas con árboles más grandes para observar el rendimiento de los recorridos.
- Mantener una estructura ordenada del código utilizando funciones independientes y comentarios descriptivos.
- Aplicar estructuras de datos como árboles en proyectos reales para fortalecer la lógica de programación y el análisis de sistemas.

2.10 Repositorio GitHub

<https://github.com/Ali-20-l/Tarea2.git>